

mdspan.at()

Document #: P3383R0
Date: 2024-09-05
Project: Programming Language C++
Audience: Library Evolution Working Group (LEWG)
SG23 Safety and Security
Reply-to: Stephan Lachnit
<stephan.lachnit@cern.ch>
Xavier Bonaventura (BMW)
<xavier.bonaventura@bmw.de>

Contents

- [1 Introduction](#)
- [2 Motivation](#)
 - [2.1 Safety](#)
 - [2.2 Consistency](#)
- [3 Impact On the Standard](#)
- [4 Wording](#)
- [5 Reference Implementation](#)
- [6 References](#)

§ 1 Introduction

This paper proposes element access with bounds checking to `std::mdspan` via `at()` member functions.

§ 2 Motivation

§ 2.1 Safety

The new `at()` member functions provide memory-safe element access to `std::mdspan`, and thus have defined behavior. Out-of-bound access can be caught by catching the `std::out_of_range` exception.

§ 2.2 Consistency

In [P2821R4], element access with bounds checking via `at()` has been added to `std::span`. One of the main motivations for this change was consistency with other containers that have element access with bounds checking via `at()`. Similarly, such element access should be added to `std::mdspan`.

§ 3 Impact On the Standard

The impact of this proposal on the standard is low. The proposed function signatures for the `at()` member functions are identical to the function signatures for the subscript operators as proposed in [P0009R18].

One consideration is that the `at()` method has previously not been used with multi-dimensional arguments. However, this was also true for the subscript operator before the possibility was introduced in [P2128R6].

§ 4 Wording

The wording is relative to [N4981].

In 17.3.2 ([[version.syn](#)]), add:

```
#define __cpp_lib_mdspan_at YYYYMMML // also in <mdspan>
```

Adjust the placeholder value as needed to denote this proposal's date of adoption.

In 24.7.3.6.1 ([[mdspan.mdspan.overview](#)]), add the following immediately after the subscript operators:

```
template<class... OtherIndexTypes>
    constexpr reference at(OtherIndexTypes... indices) const;
template<class OtherIndexType>
    constexpr reference at(span<OtherIndexType, rank()> indices)
        const;
template<class OtherIndexType>
    constexpr reference at(const array<OtherIndexType, rank()>&
        indices) const;
```

In 24.7.3.6.3 ([[mdspan.mdspan.members](#)]), add the following immediately after the subscript operators:

```
template<class... OtherIndexTypes>
    constexpr reference at(OtherIndexTypes... indices) const;
```

⁷ Constraints:

- (7.1) — (`is_convertible_v<OtherIndexTypes, index_type> && ...`) is true,
- (7.2) — (`is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...`) is true, and

(7.3) — `sizeof...(OtherIndexTypes) == rank()` is true.

8 *Returns:* `(*this)[indices...]`

9 *Throws:* `out_of_range` if
`indices_v[i] >= extent(i) || indices_v[i] < 0`
for any `indices_v[i]` in `vector<OtherIndexTypes>({indices...})`.

```
template<class OtherIndexType>
    constexpr reference at(span<OtherIndexType, rank()> indices)
        const;

template<class OtherIndexType>
    constexpr reference at(const array<OtherIndexType, rank()>&
        indices) const;
```

10 *Constraints:*

(10.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true,
and

(10.2) — `is_nothrow_constructible_v<index_type, const
OtherIndexType&>` is true.

11 *Returns:* `(*this)[indices]`

12 *Throws:* `out_of_range` if
`indices[i] >= extent(i) || indices[i] < 0`
for any `indices[i]` in `indices`.

§ 5 Reference Implementation

The `at()` member functions have been implemented in the `std::mdspan` reference implementation from the Kokkos project at Sandia National Laboratories [[kokkos/mdspan](https://github.com/kokkos/mdspan)], see [[kokkos/mdspan#302](https://github.com/kokkos/mdspan/pull/302)].

§ 6 References

[[kokkos/mdspan](https://github.com/kokkos/mdspan)] Reference implementation of `mdspan`.

<https://github.com/kokkos/mdspan>

[[kokkos/mdspan#302](https://github.com/kokkos/mdspan/pull/302)] Add element access via `at()` to `std::mdspan`.

<https://github.com/kokkos/mdspan/pull/302>

[N4981] Thomas Köppe. 2024-04-16. Working Draft, Programming Languages — C++.

<https://wg21.link/n4981>

[P0009R18] Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. 2022-07-13. MDSPAN.

<https://wg21.link/p0009r18>

[P2128R6] Corentin Jabot, Isabella Muerte, Daisy Hollman, Christian Trott, Mark Hoemmen. 2021-09-14. Multidimensional subscript operator.

<https://wg21.link/p2128r6>

[P2821R4] Jarrad J. Waterloo. 2023-07-26. `span.at()`.

<https://wg21.link/p2821r4>